【物聯網概論】課程期末專題

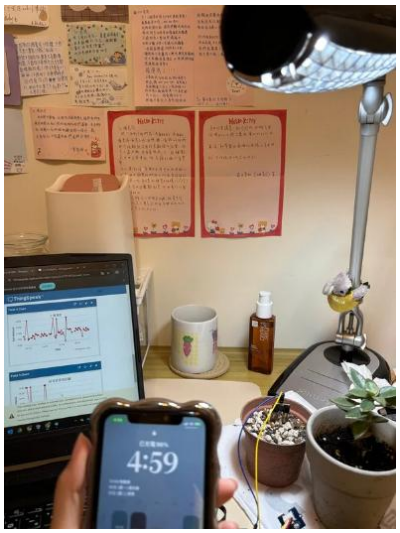
第七組

智慧植物監測系統

學號	姓名	貢獻度	簽名
412570182	閻宇虹	25%	閻宇虹
412570235	陳亮筠	25%	陳亮筠
412570376	周映妤	25%	周映妤
412570429	顏詩蓉	25%	顏詩蓉

壹、題目說明

許多人因忙碌的生活與工作，無法定期照顧家中的植物，常導致植物枯萎或健康受損，讓綠化生活變得困難。傳統的植物養殖方式主要依賴人工澆水、施肥及環境監測，但這種方式容易因人為疏忽或缺乏植物需求的知識而影響植物的健康成長。為了解決這些問題，我們計畫結合感測模組與數據分析技術，開發出一個智慧植物監測系統。該系統能即時監測植物的生長環境，例如土壤濕度、光照強度及溫度等，並透過數據分析提供圖表趨勢以供觀察。



貳、解法說明及工具

我們設計了一個智慧化系統，能夠自動監測植物的生長環境，並提供即時的反饋和建議。此系統結合多種感測模組，如溫度、濕度、光照強度和土壤濕度等數據來源，通過數據分析技術實現對植物需求的精確判斷。系統不僅能提醒使用者進行適時的澆水或環境調整，還具備可視化數據呈現的功能，讓使用者能清楚了解植物的狀態

工具使用:

1.土壤濕度感測器:

- 監測土壤的濕度狀況，確保植物獲得適量的水分供應。
- 當土壤濕度過低時，系統會亮紅燈提醒，避免植物因缺水而枯萎。

2.光線感測器:

- 監測光線強度，以確保植物獲得充足的光照

3.溫濕度感測器　:

- 監測環境溫度與濕度，確保植物生長環境適宜。

4.超音波感測器::

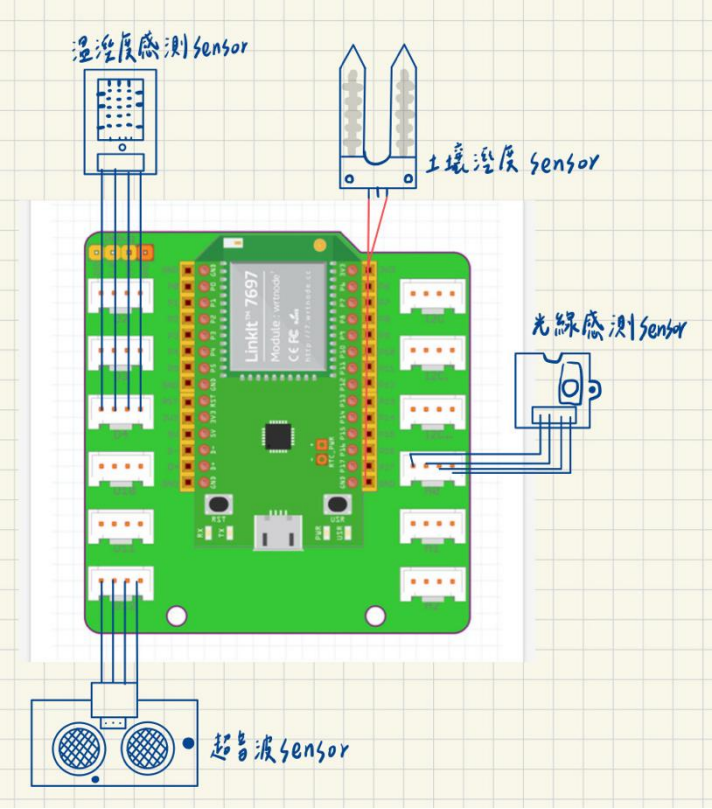
- 測量動物與植物的距離，分別為小於 10、10-20、大於 20 等監測及紅燈亮度反饋，監事職務是否受到其他生物破壞。

5.Wi-Fi 模組:

- 提供無線網路連接，讓設備能夠與互聯網通信。將感測器收集的數據上傳到雲端（如 ThingSpeak 平台），實現設備的遠程監控與操作，是物聯網應用的核心通訊工具

6.ThingSpeak:

- 用於數據的存儲、可視化和分析。它接收來自 WiFi 模組上傳的數據，並將其轉換為易於理解的圖表或儀表盤，方便用戶監控系統狀態。



參、程式碼解說及測試結果

#include "Ultrasonic.h" // 超音波感測器的相關庫 #include <DHT.h> // DHT 溫濕度感測器庫 #include <LWiFi.h> // Wi-Fi 連線功能庫

// 感測器接腳定義 #define DHTPIN 4 // DHT 溫濕度感測器的接腳 #define DHTTYPE DHT22 // DHT 感測器型號 #define LIGHT_SENSOR_PIN A0 // 光感測器接腳 #define SOIL_MOISTURE_PIN A1 // 土壤濕度感測器接腳 #define TRIGGER_PIN 12 // 超音波感測器 Trigger 接腳 #define ECHO_PIN 13 // 超音波感測器 Echo 接腳 #define LED_BUILTIN 13 // 板上內建 LED #define LED_PIN_RED 7 // 紅色警示 LED 接腳

// ThingSpeak 相關設定 #define HOST "api.thingspeak.com" #define PORT 80 char ssid[] = "iPhone"; // Wi-Fi 名稱 char pass[] = "ruby0907"; // Wi-Fi 密碼 String apiKey = "SLRH8CI4CUCKDKYX"; // ThingSpeak API 金鑰 WiFiClient client; // Wi-Fi 用戶端物件

DHT dht(DHTPIN, DHTTYPE); // 建立 DHT 感測器物件

void setup() { // 初始化序列埠通訊 Serial.begin(9600);

```
// 初始化 DHT 感測器
dht.begin();

// 設定感測器與 LED 的接腳模式
pinMode(LIGHT_SENSOR_PIN, INPUT);
pinMode(SOIL_MOISTURE_PIN, INPUT);
pinMode(LED_BUILTIN, OUTPUT);
pinMode(LED_PIN_RED, OUTPUT);
pinMode(TRIGGER_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

// 確保初始狀態 LED 關閉
digitalWrite(LED_BUILTIN, LOW);
digitalWrite(LED_PIN_RED, LOW);

// 嘗試連線 Wi-Fi
while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
```

```
    delay(500);
    Serial.print(".");
}
Serial.println("\nWi-Fi connected"); // 成功連線 Wi-Fi
printWifiStatus();                   // 顯示 Wi-Fi 連線狀態
Serial.println("System started, sensor testing...");


}
```

void loop() { // 1. 讀取感測器數據 float humidity = dht.readHumidity(); // 讀取濕度 float temperature = dht.readTemperature(); // 讀取溫度 int lightLevel = analogRead(LIGHT_SENSOR_PIN); // 讀取光線強度 int soilMoisture = analogRead(SOIL_MOISTURE_PIN); // 讀取土壤濕度

```
// 2. 超音波距離測量
long distance = measureDistance(); // 呼叫函數測量距離
if (distance == -1) {              // 如果測量失敗
    Serial.println("Ultrasonic sensor failed to detect distance.");
} else {                           // 如果測量成功，顯示距離
    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");
}

// 3. 檢查 DHT 感測器數據是否有效
if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor.");
    return; // 結束本次迴圈
}

// 4. 顯示感測器數據
Serial.print("Temperature: ");
Serial.print(temperature);
```

```
Serial.println(" °C");
Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");
Serial.print("Light Intensity: ");
Serial.println(lightLevel);
Serial.print("Soil Moisture: ");
Serial.println(soilMoisture);

// 5. 土壤濕度警報
if (soilMoisture < 500) { // 如果土壤濕度過低
    Serial.println("Warning: Low soil moisture.");
    digitalWrite(LED_BUILTIN, HIGH); // 開啟內建 LED
    digitalWrite(LED_PIN_RED, HIGH); // 開啟紅色 LED
} else {
    digitalWrite(LED_BUILTIN, LOW);  // 關閉內建 LED
    digitalWrite(LED_PIN_RED, LOW); // 關閉紅色 LED
    Serial.println("Soil moisture normal.");
}

// 6. 超音波距離反饋
handleUltrasonicFeedback(distance);

// 7. 上傳數據至 ThingSpeak
uploadToThingSpeak(humidity, temperature, lightLevel, soilMoisture,
distance);

delay(15000); // 每 15 秒更新一次


}

// 測量距離的函數 long measureDistance() { // 發送超音波脈衝
digitalWrite(TRIGGER_PIN, LOW); delayMicroseconds(2); digitalWrite(TRIGGER_PIN,
```

HIGH); delayMicroseconds(10); digitalWrite(TRIGGER_PIN, LOW);

```
// 等待回波信號，超時 30 毫秒
long duration = pulseIn(ECHO_PIN, HIGH, 30000);
if (duration == 0) {
    return -1; // 測量失敗
}

// 計算距離（cm），並過濾超出範圍的數據
long distance = duration / 58.0;
if (distance < 2 || distance > 400) {
    return -1; // 無效距離
}
return distance; // 返回有效距離


}
```

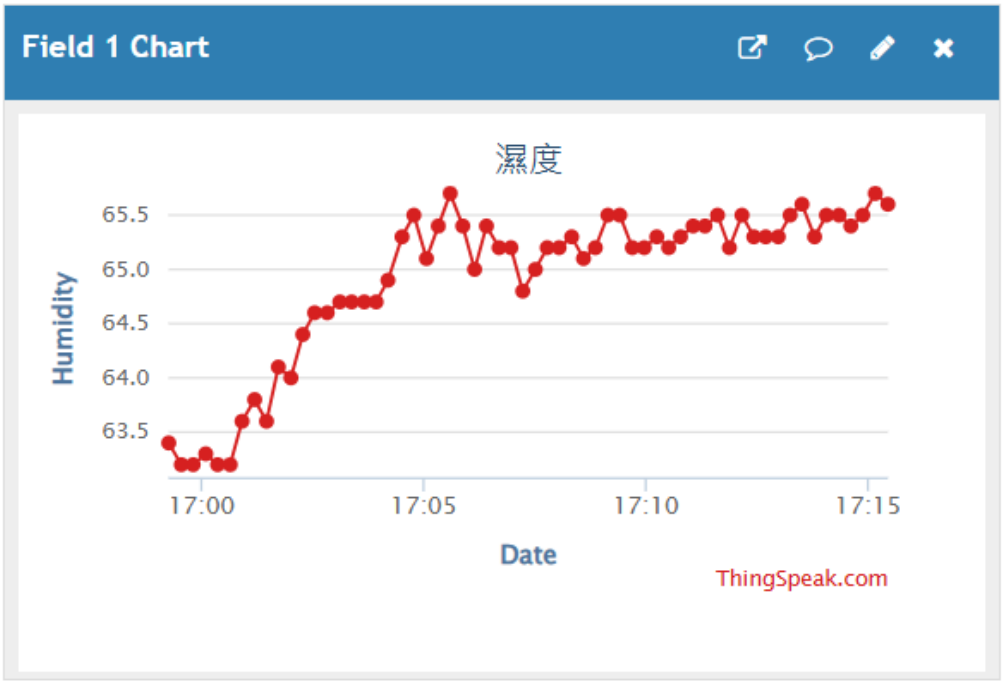
// 超音波距離反饋處理 void handleUltrasonicFeedback(int distance) { if (distance < 10 && distance > 0) { // 距離小於 10cm Serial.println("Close distance detected, fast LED blinking."); for (int i = 0; i < 3; i++) { digitalWrite(LED_BUILTIN, HIGH); delay(200); digitalWrite(LED_BUILTIN, LOW); delay(200); } } else if (distance >= 10 && distance < 20) { // 距離介於 10cm ~ 20cm Serial.println("Moderate distance detected, slow LED blinking."); digitalWrite(LED_BUILTIN, HIGH); delay(500); digitalWrite(LED_BUILTIN, LOW); delay(500); } else if (distance >= 20) { // 距離大於等於 20cm Serial.println("Distance normal, LED off."); digitalWrite(LED_BUILTIN, LOW); } else { Serial.println("No data from ultrasonic sensor."); // 無數據 } }

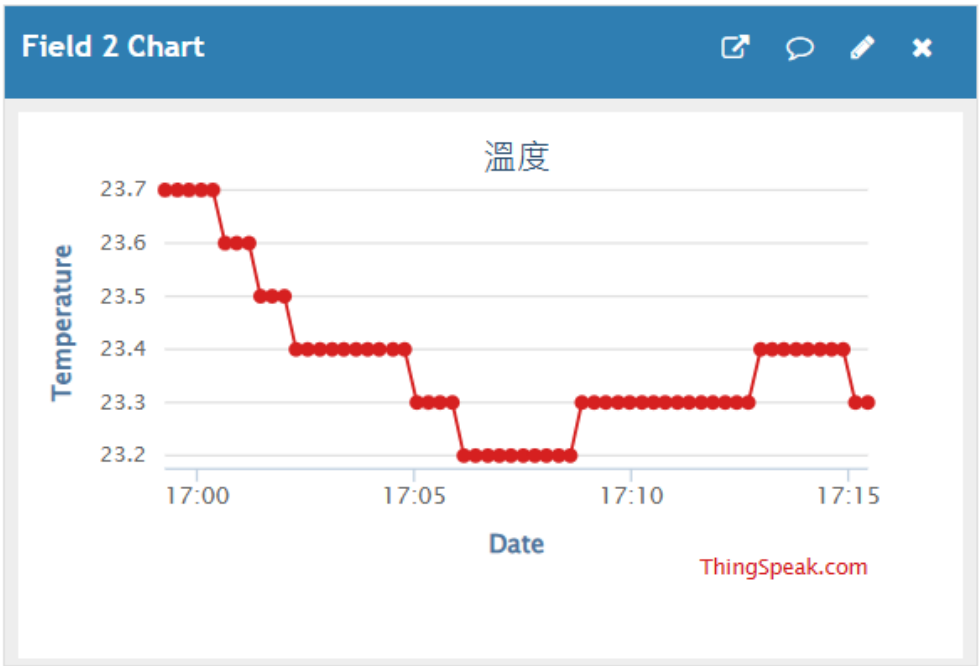
// 上傳數據至 ThingSpeak 的函數 void uploadToThingSpeak(float humidity, float temperature, int lightLevel, int soilMoisture, int distance) { if (client.connect(HOST, PORT)) { String getStr = String("GET /update?api_key=") + apiKey + "&field1=" + String(humidity) + "&field2=" + String(temperature) + "&field3=" + String(lightLevel) + "&field4=" + String(soilMoisture) + "&field5=" + String(distance) + " HTTP/1.1\r\nHost: " + HOST + "\r\nConnection: close\r\n\r\n"; client.print(getStr); // 傳送數據 Serial.println("Data sent to ThingSpeak"); while (client.connected()) { while (client.available()) { char c = client.read(); // 讀

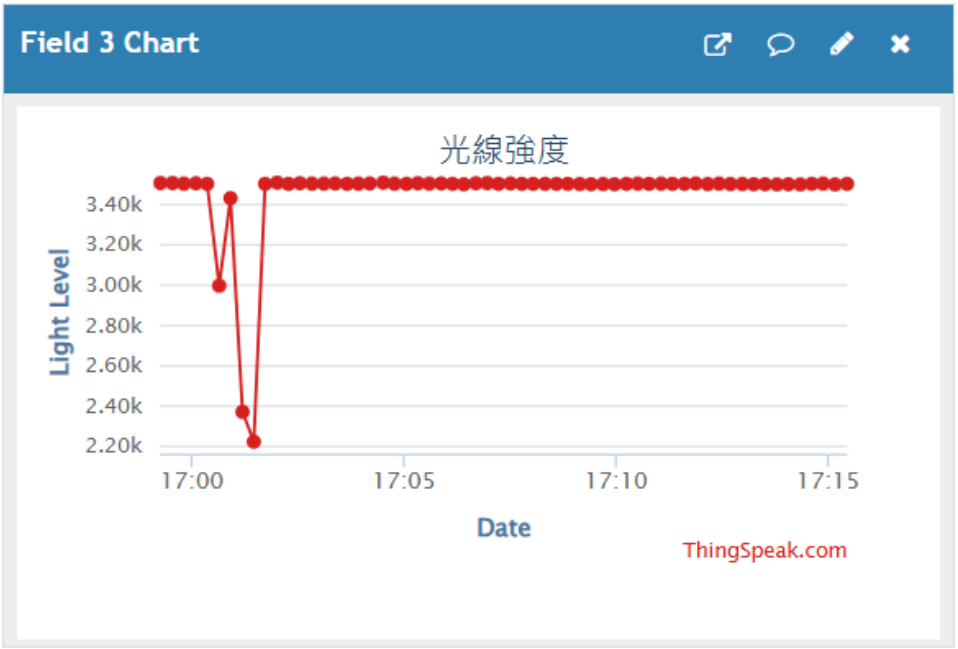
取伺服器回應 Serial.print(c); } } client.stop(); // 關閉連線 } else { Serial.println("Failed to connect to ThingSpeak."); } }

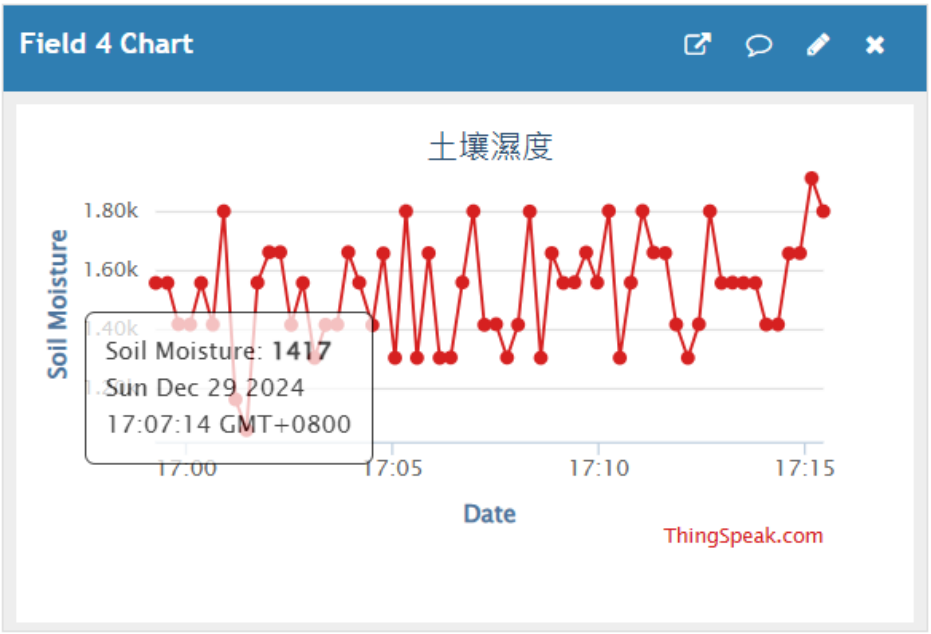
// 顯示 Wi-Fi 狀態的函數 void printWifiStatus() { Serial.print("SSID: "); Serial.println(WiFi.SSID()); IPAddress ip = WiFi.localIP(); Serial.print("IP Address: "); Serial.println(ip); long rssi = WiFi.RSSI(); Serial.print("Signal strength (RSSI): "); Serial.print(rssi); Serial.println(" dBm"); }

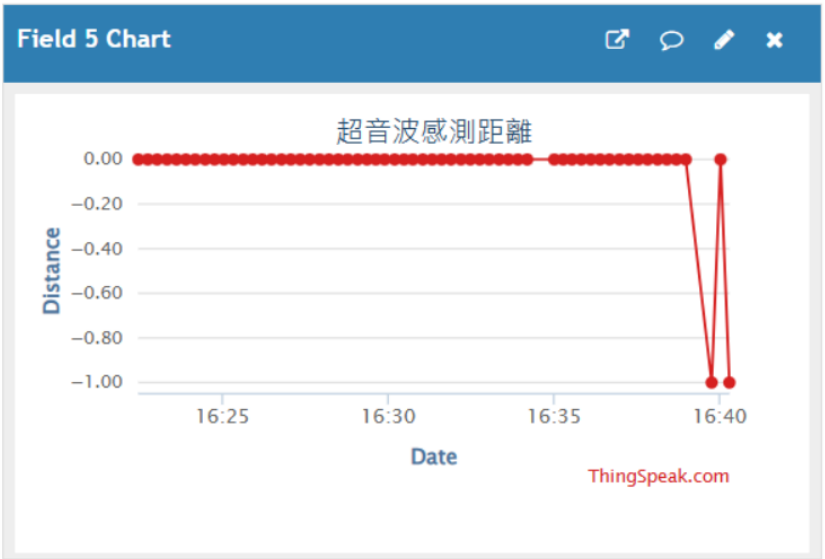
測試結果:


Field 1 Chart
濕度
Humidity
65.5
65.0
64.5
64.0
63.5
17:00
17:05
17:10
17:15
Date
ThingSpeak.com


Field 2 Chart
溫度
Temperature
23.7
23.6
23.5
23.4
23.3
23.2
17:00
17:05
17:10
17:15
Date
ThingSpeak.com


Field 3 Chart
光線強度
Light Level
3.40k
3.20k
3.00k
2.80k
2.60k
2.40k
2.20k
17:00
17:05
17:10
17:15
Date
ThingSpeak.com


Field 4 Chart
土壤濕度
Soil Moisture
1.80k
1.60k
Soil Moisture: 1417
Sun Dec 29 2024
17:07:14 GMT+0800
17:05
17:10
17:15
Date
ThingSpeak.com



肆、遇到的問題及解決方式

一、超音波感測器距離一直為 0cm

1. 困難描述：
    - 測試超音波感測器時，距離數據始終顯示為 0 cm，無法正常讀取測量值。
    - 在教室使用電腦測試時，我們嘗試更換感測器的電線後，成功讀取到正確的距離數據。
    - 然而，使用自己的電腦重新測試時，即使更換相同的電線並檢查接腳，感測器依然無法運作，距離始終顯示為 0 cm。
2. 可能原因分析：
    - 教室與家中的測試環境存在差異，包括硬體的穩定性與軟體開發環境的設定。
    - 家中電腦的 Arduino IDE 或相關函式庫版本可能與教室電腦不同，導致感測器運行異常。
3. 嘗試解決方法：
    - 更換電線與硬體檢查：確認感測器接腳與電線無損壞。
    - 軟體環境排查：檢查家中電腦的 Arduino IDE 版本，重新安裝，並同步教室環境的設定。

- 測試範例程式碼：使用函式庫提供的範例程式碼測試感測器，但仍無法成功讀取數據。

4. 最終結果：
    - 即使更換電線與重建軟體開發環境，家中電腦仍無法使超音波感測器正常運作。受限於工具與設備，無法進一步排查。
5. 學習與改進：
    - 在教室環境中成功運行的經驗說明，感測器本身是可行的，但需要更穩定的開發環境支持。

二、無法將數據上傳至 Thingspeak

1. 問題描述：
    - 在測試將感測器數據上傳至 ThingSpeak 時，無法在平台上顯示。
    - 初步確認 Wi-Fi 已連接，但不清楚如何正確配置 Wi-Fi 連接與上傳程式。
2. 解決過程與嘗試：
    - 確認 Wi-Fi 連線：檢查 Wi-Fi SSID 和密碼，並透過序列埠打印連線資訊。
    - 請教 ChatGPT：在多次嘗試無果後，詢問 ChatGPT 如何解決數據上傳問題。ChatGPT 指出可能需要檢查 ThingSpeak 的 HTTP 請求格式，並提供了範例程式作為參考。
    - 重新撰寫 HTTP 請求：根據 ChatGPT 的建議，重新撰寫程式碼以符合 ThingSpeak 的要求。確保數據正確對應至 ThingSpeak 的 field1 和 field2 欄位。
3. 最終結果：
    - 在 ChatGPT 的指導下，成功將感測器數據上傳至 ThingSpeak，並實現穩定的數據更新。

4. 學習經驗：
    - 在線技術支援的高效性： ChatGPT 提供的解決方案快速指引我們定位問題並修正程式碼。
    - 精確的 HTTP 請求格式： ThingSpeak 的數據上傳高度依賴正確的 API Key 和請求結構。
    - 逐步測試的重要性： 善用序列埠輸出檢查每個環節，有助於迅速排查問題。

伍、心得

我們這組在進行智慧植物監測系統的過程中，遇到了不少挑戰，但也獲得了寶貴的經驗。首先，在使用超音波感測器時，我們發現測量距離總是顯示為 0 cm。後來在教室電腦上嘗試更換電線後，感測器成功運作，但回家使用自己的電腦時卻又無法執行，這讓我們了解到硬體設備與環境之間可能存在的差異性，並提醒我們在不同環境中進行測試的重要性。

其次，我們在嘗試將數據上傳至 ThingSpeak 時也面臨了困難。雖然 Wi-Fi 連線已成功，但數據始終無法顯示在平台上。經過多次測試與調整，我們向 ChatGPT 請教，學習到了正確的 HTTP 請求結構與上傳格式。修改程式後，成功將感測數據上傳至 ThingSpeak，並在平台上實現即時顯示，這對我們而言是一個很大的突破。

這次專題的過程，讓我們體會到解決問題時需要耐心和創意，也讓我們更熟悉感測器與物聯網平台的整合操作。雖然部分問題尚未完全解決，但整體過程不僅提升了我們的技術能力，也增強了團隊合作與面對挑戰的信心，為未來專題奠定了更穩固的基礎。